

Chapter 4: Identifiers in Hibernate/JPA

- Introduction
- Simple Identifiers
- Generated Identifiers
- Custom Generator
- Composite Identifiers
- Derived Identifiers

Introduction

- Identifiers in Hibernate represent the primary key of an entity. This implies the values are unique so that they can identify a specific entity, that they aren't null and that they won't be modified.
- Hibernate provides a few different ways to define identifiers.

Simple Identifiers

- The most straightforward way to define an identifier is by using the @Id annotation.
- Simple ids are mapped using @Id to a single property of one of these types: Java primitive and primitive wrapper types, String, Date, BigDecimal, BigInteger.

Let's see a quick example of defining an entity with a primary key of type long:

```
@Entity
public class Student {
    @Id
    private long studentId;
    // standard constructor, getters, setters
}
```

Generated Identifiers

- If we want the primary key value to be generated automatically for us, we can add the @GeneratedValue annotation.
- This can use 4 generation types: AUTO, IDENTITY, SEQUENCE, TABLE.
- If we don't specify a value explicitly, the generation type defaults to AUTO.

AUTO Generation

- If we're using the default generation type, the persistence provider will determine values based on the type of the primary key attribute.
- This type can be numerical or UUID.
- For numeric values, the generation is based on a sequence or table generator, while UUID values will use the UUIDGenerator.

Let's see an example of mapping an entity primary key using AUTO generation strategy:

```
@Entity
public class Student {
    @Id
    @GeneratedValue
    private long studentId;
    // ...
}
```

In this case, the primary key values will be unique at the database level.

An interesting feature introduced in Hibernate 5 is the UUIDGenerator. To use this, all we need to do is declare an id of type UUID with @GeneratedValue annotation:

```
@Entity
public class Course {
    @Id
    @GeneratedValue
    private UUID courseId;
    // ...
}
```

Hibernate will generate an id of the form "8dd5f315-9788-4d00-87bb-10eed9eff566".

IDENTITY Generation

- This type of generation relies on the IdentityGenerator which expects values generated by an identity column in the database, meaning they are auto-incremented.
- To use this generation type, we only need to set the strategy parameter:

```
@Entity
public class Student {
    @Id
    @GeneratedValue (strategy = GenerationType.IDENTITY)
    private long studentId;
    // ...
}
```

One thing to note is that IDENTITY generation disables batch updates.

SEQUENCE Generation

- To use a sequence-based id, Hibernate provides the SequenceStyleGenerator class.
- This generator uses sequences if they're supported by our database, and switches to table generation if they aren't.

To customize the sequence name, we can use the @GenericGenerator annotation with SequenceStyleGenerator strategy:

```
@Entity
public class User {
    @Id
    @GeneratedValue(generator = "sequence-generator")
    @GenericGenerator(
        name = "sequence-generator",
        strategy = "org.hibernate.id.enhanced.SequenceStyleGenerator",
        parameters = {
            @Parameter(name = "sequence_name", value = "user_sequence"),
            @Parameter(name = "initial_value", value = "4"),
            @Parameter(name = "increment_size", value = "1")
        }
    )
    private long userId;
```

Compiled by Rohit Ahuja

```
// ...
}
```

In this example, we've also set an initial value for the sequence, which means the primary key generation will start at 4. SEQUENCE is the generation type recommended by the Hibernate documentation.

The generated values are unique per sequence. If you don't specify a sequence name, Hibernate will re-use the same hibernate_sequence for different types.

TABLE Generation

- The TableGenerator uses an underlying database table that holds segments of identifier generation values.

Let's customize the table name using the @TableGenerator annotation:

```
@Entity
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.TABLE,
        generator = "table-generator")
    @TableGenerator(name = "table-generator",
        table = "dep_ids",
        pkColumnName = "seq_id",
        valueColumnName = "seq_value")
    private long depId;

    // ...
}
```

In this example, we can see that other attributes such as the pkColumnName and valueColumnName can also be customized.

The disadvantage of this method is that it doesn't scale well and can negatively affect performance.

To sum up, these four generation types will result in similar values being generated but use different database mechanisms.

Custom Generator

- If we don't want to use any of the out-of-the-box strategies, we can define our custom generator by implementing the IdentifierGenerator interface.

Let's create a generator that builds identifiers containing a String prefix and a number:

```
public class MyGenerator implements IdentifierGenerator, Configurable {

    private String prefix;

    @Override
    public Serializable generate(SharedSessionContractImplementor session, Object obj)
        throws HibernateException {
```

```

String query = String.format("select %s from %s",
    session.getEntityPersister(obj.getClass().getName(), obj)
        .getIdentifierPropertyName(),
    obj.getClass().getSimpleName());

Stream ids = session.createQuery(query).stream();

Long max = ids.map(o -> o.replace(prefix + "-", ""))
    .mapToLong(Long::parseLong)
    .max()
    .orElse(0L);

return prefix + "-" + (max + 1);
}

@Override
public void configure(Type type, Properties properties, ServiceRegistry serviceRegistry) throws MappingException
{
    prefix = properties.getProperty("prefix");
}
}

```

In this example, we override the generate() method from the IdentifierGenerator interface and first find the highest number from the existing primary keys of the form prefix-XX.

Then we add 1 to the maximum number found and append the prefix property to obtain the newly generated id value.

Our class also implements the Configurable interface, so that we can set the prefix property value in the configure() method.

Next, let's add this custom generator to an entity. For this, we can use the @GenericGenerator annotation with a strategy parameter that contains the full class name of our generator class:

```

@Entity
public class Product {

    @Id
    @GeneratedValue(generator = "prod-generator")
    @GenericGenerator(name = "prod-generator",
        parameters = @Parameter(name = "prefix", value = "prod"),
        strategy = "com.vedisoft.hibernate.pojo.generator.MyGenerator")
    private String prodId;

    // ...
}

```

Also, notice we've set the prefix parameter to "prod".

```
public void whenSaveCustomGeneratedId_thenOk() {
    Product product = new Product();
    session.save(product);
    Product product2 = new Product();
    session.save(product2);
}
```

Here, the first value generated using the “prod” prefix was “prod-1”, followed by “prod-2”.

Besides the simple identifiers we've seen so far, Hibernate also allows us to define composite identifiers.

Composite Identifiers

- A composite id is represented by a primary key class with one or more persistent attributes.
- The primary key class must fulfill several conditions:
 - o it should be defined using @EmbeddedId or @IdClass annotations
 - o it should be public, serializable and have a public no-arg constructor
 - o it should implement equals() and hashCode() methods
 - o The class's attributes can be basic, composite or ManyToOne while avoiding collections and OneToOne attributes.

@EmbeddedId

To define an id using @EmbeddedId, first we need a primary key class annotated with @Embeddable:

```
@Embeddable
public class OrderEntryPK implements Serializable {

    private long orderId;
    private long productId;

    // standard constructor, getters, setters
    // equals() and hashCode()
}
```

Next, we can add an id of type OrderEntryPK to an entity using @EmbeddedId:

```
@Entity
public class OrderEntry {

    @EmbeddedId
    private OrderEntryPK entryId;

    // ...
}
```

Let's see how we can use this type of composite id to set the primary key for an entity:

```
public void whenSaveCompositeIdEntity_thenOk() {
    OrderEntryPK entryPK = new OrderEntryPK();
    entryPK.setOrderId(1L);
    entryPK.setProductId(30L);
}
```

```

OrderEntry entry = new OrderEntry();
entry.setEntryId(entryPK);
session.save(entry);
}

```

Here the OrderEntry object has an OrderEntryPK primary id formed of two attributes: orderId and productId.

@IdClass

The @IdClass annotation is similar to the @EmbeddedId, except the attributes are defined in the main entity class using @Id for each one.

The primary-key class will look the same as before.

Let's rewrite the OrderEntry example with an @IdClass:

```

@Entity
@IdClass(OrderEntryPK.class)
public class OrderEntry {
    @Id
    private long orderId;
    @Id
    private long productId;

    // ...
}

```

Then we can set the id values directly on the OrderEntry object:

```

public void whenSaveIdClassEntity_thenOk() {
    OrderEntry entry = new OrderEntry();
    entry.setOrderId(1L);
    entry.setProductId(30L);
    session.save(entry);
}

```

The disadvantage of this method is that there's no separation between the entity object and the identifier.

Derived Identifiers

Derived identifiers are obtained from an entity's association using the @MapsId annotation.

First, let's create a UserProfile entity which derives its id from a one-to-one association with the User entity:

```

@Entity
public class UserProfile {

    @Id
    private long profileId;

    @OneToOne

```

```
@MapsId
private User user;

// ...
}
```

Next, let's verify that a UserProfile instance has the same id as its associated User instance:

```
public void whenSaveDerivedIdEntity_thenOk() {
    User user = new User();
    session.save(user);
    UserProfile profile = new UserProfile();
    profile.setUser(user);
    session.save(profile);
}
```

Example: test7

Pre-Requirement: Create a database by the **hib** in **MySQL 8**

1. test7\pom.xml
2. src\main\java\hibernate.cfg.xml
3. com.vedisoft.hibernate.test7.utils
 - **HibernateUtil.java**
4. com.vedisoft.hibernate.test7.pojos
 - **Student.java** @Entity
 - **Student1.java** @Entity
 - **Course.java** @Entity
 - **Student2.java** @Entity
 - **User.java** @Entity
 - **Depratment.java** @Entity
 - **Product.java** @Entity
 - **OrderEntry.java** @Entity
 - **OrderEntryIdClass.java** @Entity
 - **UserProfile.java** @Entity
5. com.vedisoft.hibernate.test7.daos
 - **IdentifierDao.java**
 - **IdentifierDao.java**
6. com.vedisoft.hibernate.test7
 - **App.java**